

Metodología de Sistemas II

Trabajo Práctico Factory Method

Refactoring en el mundo real - Sistema de reportes con múltiples formatos

Alumno:

Matias Carro - matiasmanuelcarro@gmail.com

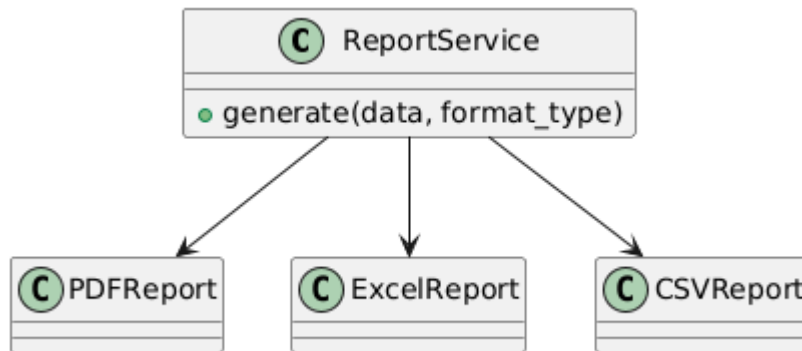
DNI: 37362003

Legajo: 18743

Diagrama UML.....	3
Código original (Antes):.....	3
Codigo Despues:.....	3
Código.....	4
Código antes (Traducido a Python):.....	4
Codigo despues (Solución en Python):.....	4
Problema resuelto:.....	8
¿Por qué Factory Method y no Abstract Factory?.....	8

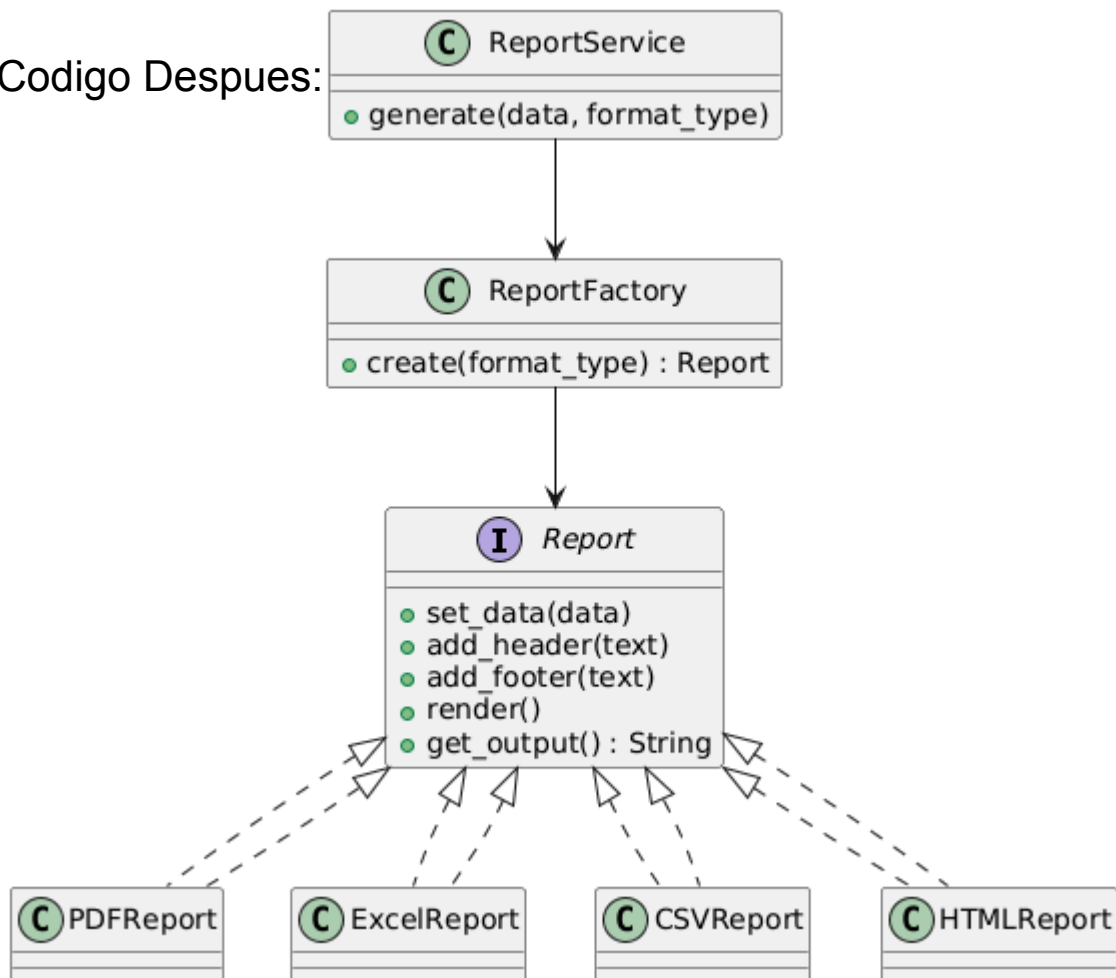
Diagrama UML

Código original (Antes):



Todo el proceso sucede dentro de la clase ReportService

Código Después:



Se aplica la interfaz Report, ReportFactory, ReportService y una clase para cada tipo de Report

Código

Código antes (Traducido a Python):

```
class ReportService:
    def generate(self, data, format_type):
        # decisión de construcción mezclada con lógica de uso
        if format_type == "pdf":
            report = PDFReport()
        elif format_type == "excel":
            report = ExcelReport()
        elif format_type == "csv":
            report = CSVReport()
        else:
            raise Exception("Formato no soportado")

        # lógica de uso - igual para todos los tipos
        report.set_data(data)
        report.add_header("Reporte Mensual")
        report.add_footer("Generado el " + hoy())
        report.render()
        return report.get_output()
```

Código después (Solución en Python):

```
from abc import ABC, abstractmethod
from datetime import date

# 1. La interfaz que todos los reportes cumplen
class Report(ABC):

    @abstractmethod
    def set_data(self, data):
        pass

    @abstractmethod
    def add_header(self, text):
```

```

        pass

    @abstractmethod
    def add_footer(self, text):
        pass

    @abstractmethod
    def render(self):
        pass

    @abstractmethod
    def get_output(self) -> str:
        pass

```

2. Las clases concretas

```

class PDFReport(Report):
    def __init__(self):
        self.content = []

    def set_data(self, data):
        self.content.append(f"PDF Data: {data}")

    def add_header(self, text):
        self.content.insert(0, f"PDF Header: {text}")

    def add_footer(self, text):
        self.content.append(f"PDF Footer: {text}")

    def render(self):
        self.content.append("PDF Rendered")

    def get_output(self) -> str:
        return "\n".join(self.content)

```

```

class ExcelReport(Report):
    def __init__(self):
        self.rows = []

    def set_data(self, data):
        self.rows.append(f"Excel Data: {data}")

```

```
def add_header(self, text):
    self.rows.insert(0, f"Excel Header: {text}")

def add_footer(self, text):
    self.rows.append(f"Excel Footer: {text}")

def render(self):
    self.rows.append("Excel Rendered")

def get_output(self) -> str:
    return "\n".join(self.rows)
```

```
class CSVReport(Report):
    def __init__(self):
        self.lines = []

    def set_data(self, data):
        self.lines.append(f"CSV Data: {data}")

    def add_header(self, text):
        self.lines.insert(0, f"CSV Header: {text}")

    def add_footer(self, text):
        self.lines.append(f"CSV Footer: {text}")

    def render(self):
        self.lines.append("CSV Rendered")

    def get_output(self) -> str:
        return "\n".join(self.lines)
```

```
class HTMLReport(Report):
    def __init__(self):
        self.html = []

    def set_data(self, data):
        self.html.append(f"HTML Data: {data}")

    def add_header(self, text):
```

```

        self.html.insert(0, f"HTML Header: {text}")

    def add_footer(self, text):
        self.html.append(f"HTML footer: {text}")

    def render(self):
        self.html.append("HTML Rendered")

    def get_output(self) -> str:
        return "\n".join(self.html)

```

3. La fábrica concentra la decisión de construcción

```

class ReportFactory:
    @staticmethod
    def create(format_type: str) -> Report:
        if format_type == "pdf":
            return PDFReport()
        elif format_type == "excel":
            return ExcelReport()
        elif format_type == "csv":
            return CSVReport()
        elif format_type == "html":
            return HTMLReport()
        else:
            raise ValueError("Formato no soportado")

```

4. El servicio ahora es agnóstico al formato

```

class ReportService:
    def generate(self, data, format_type):
        report = ReportFactory.create(format_type) # no sabe qué tipo es
        report.set_data(data)
        report.add_header("Reporte Mensual")
        report.add_footer("Generado el " + str(date.today()))
        report.render()
        return report.get_output()

```

Ejemplo de uso

```

servicio = ReportService()
print(servicio.generate("Datos en html", "html"))

```

```

print("=====")
print(servicio.generate("Datos de csv", "csv"))

# Se uso Factory Method porque alcanza con una fábrica que decida
# qué tipo de reporte crear. El servicio no necesita saber si es PDF,
# Excel o CSV, solo lo usa. Esto evita acoplar el código y permite
# agregar nuevos formatos sin tocar la lógica del servicio.

```

Test del programa:

```

HTML Header: Reporte Mensual
HTML Data: Datos de prueba
HTML footer: Generado el 2026-08-28
HTML Rendered
=====
CSV Header: Reporte Mensual
CSV Data: Datos de csv
CSV Footer: Generado el 2026-08-28
CSV Rendered

```

Problema resuelto:

El problema que resolvió el patrón fue el acoplamiento entre el servicio y las clases de reporte. Antes, el código estaba unificado en `ReportService` e instanciaba directamente cada tipo de reporte, lo que era código rígido y difícil de mantener. Cada vez que se agregaba un nuevo formato había que modificar la lógica del servicio y agregar condicionales.

Con el Factory Method, la creación de objetos quedó aislada en un único lugar y el servicio solo se ocupa del uso. Esto hace que el método `generate()` que usan los demás clases para pedir un reporte nunca cambie, además permite incorporar nuevos formatos como `HTMLReport` sin tocar el servicio y lograr que el sistema sea más extensible y fácil de mantener.

¿Por qué Factory Method y no Abstract Factory?

En este ejercicio no era necesario usar Abstract Factory porque ese patrón se aplica cuando hay que crear familias de objetos relacionados entre sí, que comparten un mismo estilo. En este caso solo tenemos un único tipo de objeto (reportes) en distintas formas (PDF, Excel, CSV, HTML) pero que siguen siendo un solo tipo de objeto, un reporte. Por eso, el problema se resolvió con Factory Method, que es suficiente para encapsular la construcción de un solo objeto y permitir que el servicio trabaje de manera independiente de las distintas clases.

